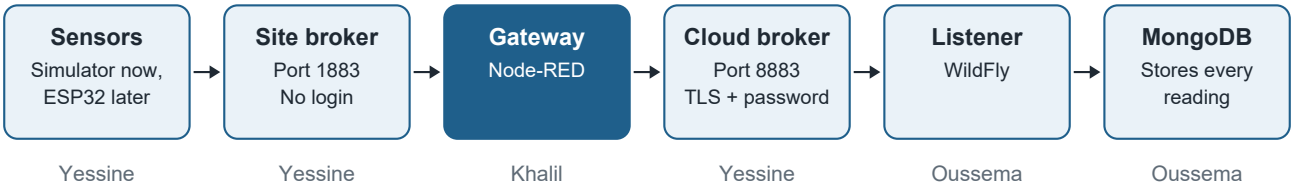
The ColdTrace IoT gateway

What it does, and what must connect to it so the work can continue · Nov 2, 2026

In one minute

The gateway is the **post office between the sensors and the cloud**. Sensors hand it their readings. It throws away the broken ones, sends the good ones to the cloud over a secure connection, and holds on to them if the internet is down.



The name under each box shows who owns that part in the work plan. Khalil owns the gateway. Yessine connects on the left side, and Oussema connects on the right side.

What the gateway does

Job	In easy words
Receive	Listens for readings from all sensors on the site broker.
Check	Rejects bad data: not JSON, no measurements, or impossible values. Temperature must be −40 to 85 °C, humidity 0 to 100 %.
Clean up	Rounds values to 0.1, fixes missing or wrong times, and gives each reading a unique ID.
Send safely	Sends good readings to the cloud broker using TLS and a password.
Keep when offline	If the cloud is down, saves up to 20,000 readings on disk and sends them when it comes back, even after a restart. With 10 sensors every 5 seconds, that covers about 2.8 hours.
Report its health	Tells the cloud every minute that it is alive, with counts of sent, rejected and waiting readings.
Accept new protocols	MQTT is the only one active today, but another protocol can be added with a small adapter. The rest of the gateway and the messages sent to the cloud stay the same.

What it does not do: it does not create alerts, save to MongoDB, or show anything to users. If a sensor goes silent, the gateway does not report it either. All of that belongs to the back-end.

Where it stands today

- **Written:** the Node-RED flow, both brokers, TLS certificates and a small 3-sensor simulator.
- **Not on GitHub yet:** the folder still has to be committed and sent as a pull request.
- **Not re-tested:** the adapter layer for new protocols has not been run yet, so the flow must be tested again.
- **Not started:** the ESP32 firmware and the Raspberry Pi setup (Phase 3).

Yessine: what connects on the sensor side

The simulator only has to publish messages in this format to the site broker. Nothing else is needed for the gateway to accept them.

```
Broker:  site-broker:1883 (localhost:1883 from your laptop), no login, QoS 1
Topic:   coldtrace/{siteId}/{sensorId}/telemetry      example: coldtrace/site02/S07/telemetry
Message: { "ts": 1793632152, "temperature": -18.4, "humidity": 61.5, "doorOpen": false }
```

- The site and sensor IDs go in the topic, not the message. IDs use letters, digits, - or _, up to 32 characters.
- Each measurement is optional, but at least one must be there. `ts` is optional too and can be Unix time.
- Use the scenarios from the plan freely: door left open and rising temperature pass through as normal readings. Sensor gone silent simply means no messages.
- **Please build on the existing files.** `gateway-iot/docker-compose.yml` and `simulator/` already exist. Add MongoDB to that compose file and extend the simulator to 10 sensors on 3 sites, so we do not end up with two setups.
- **Keep both brokers.** The plan mentions one Mosquitto, but the gateway uses the site broker for sensors and the cloud broker for the back-end.

Oussema: what connects on the back-end side

The listener subscribes to the cloud broker and receives clean readings. It never talks to sensors or to Node-RED directly.

Setting	Value
Address	localhost:8883 from your laptop, or cloud-broker:8883 inside the same Docker network
Security	TLS. Trust `certs/ca.crt`. The certificate is valid only for localhost, 127.0.0.1 and cloud-broker; add other names with `gen-certs.sh <name>`
Login	User `backend`, password `CLOUD_BACKEND_PASSWORD` from `.env` (never commit it)
Readings topic	`coldtrace/+/+/telemetry`
Gateway health topic	`coldtrace/+/gateway/status`
Connection	QoS 1, a fixed client ID, and clean session off, so readings wait for you if WildFly restarts

Each reading looks like this:

```
{ "v": 1, "msgId": "site01-S01-1793632152000", "siteId": "site01", "sensorId": "S01",
  "gatewayId": "gw-site01", "ts": "2026-11-02T15:09:12.000Z", "tsSource": "sensor",
  "temperature": 4.2, "humidity": 61.5, "doorOpen": false,
  "receivedAt": "2026-11-02T15:09:12.185Z", "replayed": true }
```

Rules the listener must follow:

- **Ignore duplicates.** The same reading can arrive twice after a reconnect. Make `msgId` unique in MongoDB.
- **Accept missing measurements.** `temperature`, `humidity` and `doorOpen` can each be absent, so the `Measurement` class must allow that.
- **Use `ts` for the time of the measurement,** and sort by it. `receivedAt` is only when the gateway got it.
- **Do not treat `replayed` as "old".** It means the reading waited on the gateway, but today new readings can get it too just after an outage. Check the age from `ts`.
- **Detect silent sensors.** The gateway cannot report a sensor that stops sending, so the back-end should alert when one has not been heard from for too long.
- **Match the IDs.** `siteId` and `sensorId` must be the same IDs used by the Site and Sensor classes.

What we must decide together

1. This blocks the 3-site test: one gateway login only works for one site

The cloud broker lets a gateway write only under its own site, and the current login is `site01`. If the simulator sends 3 sites through one gateway, readings from site02 and site03 are dropped with no error, and the gateway still counts them as sent. **Suggestion:** on laptops, use one development login allowed to write for all sites, and keep one login per site on the real server. Yessine and Khalil should agree this before the simulator is extended.

2. Agree on the message format before the classes are written

The format is still a draft (`MESSAGE-FORMAT.md`). Oussema's data classes and Yessine's simulator both depend on it. Field names to confirm: `temperature`, `humidity`, `doorOpen`, `ts`. The hardware list also has two temperature sensors (DS18B20 and DHT22), but the message has only one temperature field.

Run it and check the connection

You need Docker Desktop and Git Bash. Run these from the `gateway-iot` folder. On Windows, keep the `MSYS_NO_PATHCONV=1` part; without it, creating the certificates fails.

```
cp .env.example .env                        # then change the two passwords
MSYS_NO_PATHCONV=1 ./scripts/gen-certs.sh
docker compose up -d

# Terminal 1: watch what the back-end would receive (use your backend password)
MSYS_NO_PATHCONV=1 docker compose exec cloud-broker mosquitto_sub -h cloud-broker -p 8883 \
  --cafile /mosquitto/certs/ca.crt -u backend -P YOUR_PASSWORD -t 'coldtrace/#' -v

# Terminal 2: send one reading by hand
docker compose exec site-broker mosquitto_pub -h localhost -q 1 \
  -t coldtrace/site01/S01/telemetry -m '{"temperature":4.2,"humidity":61.5,"doorOpen":false}'
```

Terminal 1 should show the reading in the cloud format. The Node-RED editor is at http://localhost:1880.

Connected means

- The simulator's readings from all 3 sites arrive on the cloud broker.
- Broken messages are rejected and never reach the cloud.
- Each good reading is saved once in MongoDB, even after the cloud broker is stopped and restarted.
- A freezer reading at −18 °C and a door-open reading both arrive as sent.

Known gateway issues Khalil will fix

You do not need to work around these, but they could explain odd results in testing.

- After a long outage, new readings wait behind the old ones, so they can be late by more than the 10-second alert target.
- If the connection drops while saved readings are being sent, a few can be lost.
- A failed DS18B20 reports 85 °C, and the gateway currently accepts it as real.
- For real ESP32s, port 1883 must be opened beyond the laptop. It is local-only today.